

Robot Handle Control

Robot Handle Control

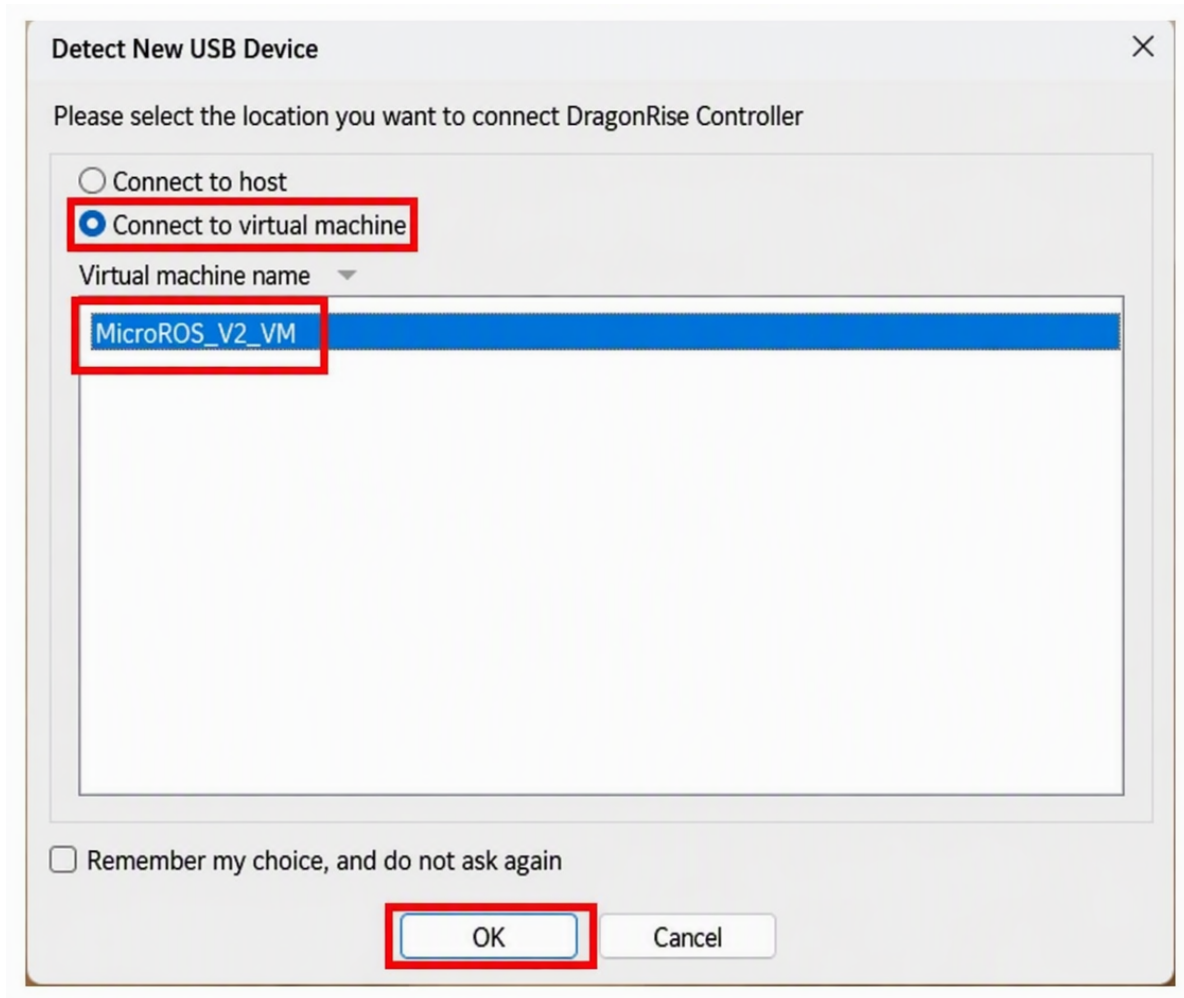
1. Course Content
2. Preparation
3. Handle Button Operations
 - 3.1 Motion Control
 - 3.2 Speed Gears
 - 3.3 Gimbal Control
 - 3.4 Buzzer and Enable Switch
4. Source Code Analysis
 - 4.1 Overall Architecture
 - 4.2 Program Execution Flow

1. Course Content

- Use a gamepad to control the robot chassis and the gimbal camera
- Understand the internal implementation source code of gamepad control

2. Preparation

- Plug the USB gamepad receiver into the computer, and choose to connect the USB device to the virtual machine



- Check that the virtual machine correctly recognizes the gamepad receiver. Usually the device is displayed as js*

```
ls /dev/input/js*
```

```
yahboom@yahboom:~$ ls /dev/input/js*  
/dev/input/js0 /dev/input/js1
```

- Start the gamepad control program

```
ros2 launch microros_v2_bringup controller.launch.py control_mode:=joystick
```

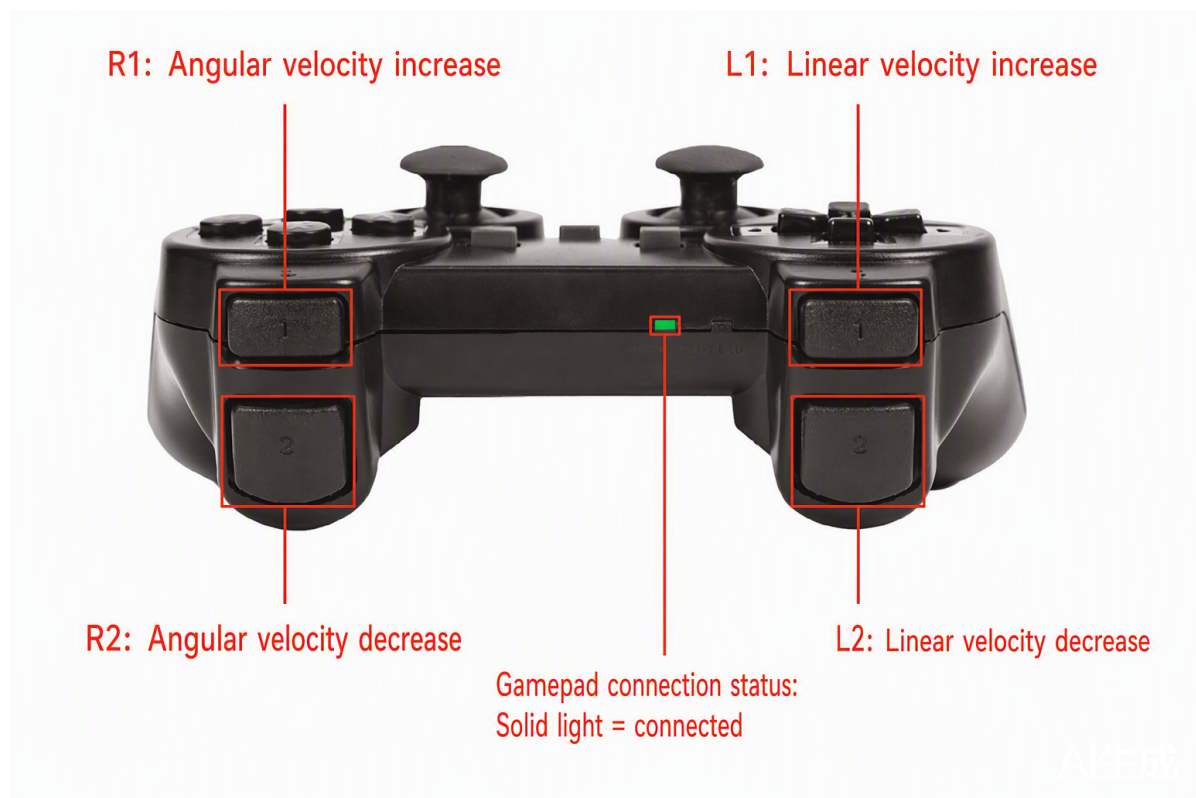
```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup controller.launch.py control_mode:=joystick
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-27-11-44-17-114963-yahboom-285693
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [285749]
[INFO] [base_driver-2]: process started with pid [285751]
[INFO] [joy_node-3]: process started with pid [285753]
[INFO] [joy_control_node-4]: process started with pid [285755]
[micro_ros_agent-1] [1785123858.039750] info      | UDPv4AgentLinux.cpp | init          | running...    | port: 8090
[base_driver-2] [INFO] [1785123858.328352775] [base_driver]: == BaseDriverNode Parameters ==
[base_driver-2] [INFO] [1785123858.328605946] [base_driver]:   odom_linear_speed      : 0.15 m/s
[base_driver-2] [INFO] [1785123858.328792702] [base_driver]:   odom_angular_speed    : 0.5 rad/s
[base_driver-2] [INFO] [1785123858.328947792] [base_driver]:   odom_position_tolerance : 0.02 m
[base_driver-2] [INFO] [1785123858.329151137] [base_driver]:   odom_angle_tolerance   : 0.05 rad
[base_driver-2] [INFO] [1785123858.329300145] [base_driver]:   odom_timeout           : 30.0 s
[base_driver-2] [INFO] [1785123858.329442197] [base_driver]:   min_linear_speed       : 0.03 m/s
[base_driver-2] [INFO] [1785123858.329588903] [base_driver]:   min_angular_speed      : 0.2 rad/s
[base_driver-2] [INFO] [1785123858.329742948] [base_driver]:   precise_adjust_xy_pid   : [0.5, 0.0, 0.05]
[base_driver-2] [INFO] [1785123858.329884456] [base_driver]:   precise_adjust_theta_pid: [1.0, 0.0, 0.3]
[base_driver-2] [INFO] [1785123858.330029782] [base_driver]:   precise_adjust_max_lin  : 0.12 m/s
[base_driver-2] [INFO] [1785123858.330168416] [base_driver]:   precise_adjust_max_ang  : 0.5 rad/s
[base_driver-2] [INFO] [1785123858.330309527] [base_driver]:   precise_adjust_pos_tol  : 0.01 m
[base_driver-2] [INFO] [1785123858.330447006] [base_driver]:   precise_adjust_ang_tol  : 0.06 rad
[base_driver-2] [INFO] [1785123858.330593010] [base_driver]: =====
[joy_node-3] [INFO] [1785123858.465679132] [joy_node]: No haptic (rumble) available, skipping initialization
[joy_node-3] [INFO] [1785123858.465798225] [joy_node]: Opened joystick: Controller.  deadzone: 0.050000

```

3. Handle Button Operations





3.1 Motion Control

Per-axis independent stick/D-pad arbitration — the stick takes priority, and the D-pad acts as a fallback only when the corresponding stick axis is idle:

Input	Function	Curve	Description
Left stick up/down (LY)	Forward / backward	Square curve (power=2.0)	Analog control, with 0.15 deadzone
Right stick left/right (RX)	Turn left / turn right	Cubic curve (power=3.0)	Analog control, with 0.15 deadzone
Left stick left/right (LX)	No function	—	Not used
Right stick up/down (RY)	No function	—	Not used
D-pad up/down	Forward / backward	—	Only active when LY is idle (digital fallback)
D-pad left/right	Turn left / turn right	—	Only active when RX is idle (digital fallback)

3.2 Speed Gears

There are 5 fixed speed gears (G1–G5), default G2:

Gear	Linear Velocity	Angular Velocity
G1	0.20 m/s	0.50 rad/s
G2 ★	0.30 m/s	1.50 rad/s
G3	0.50 m/s	2.50 rad/s
G4	0.75 m/s	3.75 rad/s
G5	1.00 m/s	5.00 rad/s

Button	Function
L1	Linear velocity up one gear
L2	Linear velocity down one gear
R1	Angular velocity up one gear
R2	Angular velocity down one gear

Moving up to G5 or down to G1 triggers a double beep.

3.3 Gimbal Control

Button	Function	Description
X	Servo S1 forward	Active when X is pressed alone
B	Servo S1 reverse	Active when B is pressed alone
Y	Servo S2 reverse	Active when Y is pressed alone
A	Servo S2 forward	Active when A is pressed alone

S1 travel range: -90° ~ +90°, S2 travel range: -90° ~ +20°.

3.4 Buzzer and Enable Switch

Button	Function
START (OPTIONS)	Press and hold to keep beeping, release to stop
SELECT (SHARE)	Toggle the gamepad enable state (/Joystate : True ↔ False)

4. Source Code Analysis

- Source code path

```
$HOME/microros_ws/src/common_demo/common_demo/control/joy_control.py
```

4.1 Overall Architecture

The node uses a **multi-timer parallel** architecture: it subscribes to the `/joy` topic to update the button/stick state, and three timers publish commands at periods of 50ms (chassis velocity), 50ms (gimbal angle), and 20ms (buzzer) respectively. `_joy_valid()` is used to uniformly determine whether the gamepad is online (0.5s timeout) and the enable switch is on.

4.2 Program Execution Flow

1. **Handle event handling (`_joy_callback`)**: After receiving a `/joy` message, it first uses `EdgeDetector` to detect the SELECT button edge, toggles the enable state and triggers a beep of the corresponding duration; when enabled, it then detects the L1/L2/R1/R2 edges for G1-G5 gear shifting — a single beep for normal shifting, and a double beep when reaching the limit.
2. **Chassis velocity computation (`_compute_target_velocity`)**: Uses "stick priority, D-pad fallback" arbitration — when the LY axis exceeds the 0.15 deadzone, it is mapped to linear velocity through a square curve; otherwise the D-pad up/down is used. When the RX axis exceeds the deadzone, it is mapped to angular velocity through a cubic curve; otherwise the D-pad left/right is used. Returns the `(target_linear, target_angular, stop)` tuple.
3. **Chassis velocity publishing (`_cmd_timer_callback`)**: When enabled normally and `stop=False`, it follows the target velocity instantly; otherwise it enters soft-stop, linearly decaying the current velocity at the deceleration rate, and forces a hard stop after `soft_stop_max_time`. Finally it clamps to the maximum linear/angular velocity and publishes a `Twist` to `/cmd_vel`.
4. **Gimbal control (`_compute_servo_direction` + `_servo_timer_callback`)**: X/B control the S1 axis forward/reverse, Y/A control the S2 axis forward/reverse; when both buttons of a pair are pressed, they cancel to 0. `_update_servo_axis` updates the angle through acceleration/deceleration interpolation toward the target velocity, and zeroes the velocity when reaching the limits (S1: $\pm 90^\circ$, S2: $-90^\circ \sim +20^\circ$); finally it packs the result into a `Vector3` and publishes it to `/cmd_ptz_angle`.

```
def _joy_callback(self, msg: Joy) -> None:
    self._latest_joy = msg
    self._last_joy_time = self.get_clock().now()
    self._has_joy = True

    # SELECT toggle works regardless of enable state
    if self._select_edge.update(self._button(msg, BTN_SELECT)):
        self._joy_control_enabled = not self._joy_control_enabled
        self.get_logger().info(f"joystick control enabled -> {self._joy_control_enabled}")
        if self._joy_control_enabled:
            self._joy_toggle_beep_ticks = 12 # enable: 12 x 20ms ≈ 250ms
        else:
            self._joy_toggle_beep_ticks = 50 # disable: 50 x 20ms = 1000ms

    if not self._joy_control_enabled:
        return

    # --- Speed gear adjustment (edge-triggered, 5 fixed gears) ---
    if self._linear_up_edge.update(self._button(msg, BTN_L1)):
        if self._linear_gear_index < 4:
            self._linear_gear_index += 1
            self.get_logger().info(
```

```

        f"linear gear -> G{self._linear_gear_index+1} "
        f"({LINEAR_GEAR_TABLE[self._linear_gear_index]:.2f} m/s)"
    )
    self._speed_beep.trigger_single()
else:
    self.get_logger().info(
        f"linear gear at max G5 ({LINEAR_GEAR_TABLE[4]:.2f} m/s)"
    )
    self._speed_beep.trigger_double()

if self._linear_down_edge.update(self._button(msg, BTN_L2)):
    if self._linear_gear_index > 0:
        self._linear_gear_index -= 1
        self.get_logger().info(
            f"linear gear -> G{self._linear_gear_index+1} "
            f"({LINEAR_GEAR_TABLE[self._linear_gear_index]:.2f} m/s)"
        )
        self._speed_beep.trigger_single()
    else:
        self.get_logger().info(
            f"linear gear at min G1 ({LINEAR_GEAR_TABLE[0]:.2f} m/s)"
        )
        self._speed_beep.trigger_double()

if self._angular_up_edge.update(self._button(msg, BTN_R1)):
    if self._angular_gear_index < 4:
        self._angular_gear_index += 1
        self.get_logger().info(
            f"angular gear -> G{self._angular_gear_index+1} "
            f"({ANGULAR_GEAR_TABLE[self._angular_gear_index]:.2f}
rad/s)"
        )
        self._speed_beep.trigger_single()
    else:
        self.get_logger().info(
            f"angular gear at max G5 ({ANGULAR_GEAR_TABLE[4]:.2f}
rad/s)"
        )
        self._speed_beep.trigger_double()

if self._angular_down_edge.update(self._button(msg, BTN_R2)):
    if self._angular_gear_index > 0:
        self._angular_gear_index -= 1
        self.get_logger().info(
            f"angular gear -> G{self._angular_gear_index+1} "
            f"({ANGULAR_GEAR_TABLE[self._angular_gear_index]:.2f}
rad/s)"
        )
        self._speed_beep.trigger_single()
    else:
        self.get_logger().info(
            f"angular gear at min G1 ({ANGULAR_GEAR_TABLE[0]:.2f}
rad/s)"
        )
        self._speed_beep.trigger_double()

# -----
# validity check – /joy freshness + enable state

```

```

# -----

def _joy_valid(self) -> bool:
    age_s = (self.get_clock().now() - self._last_joy_time).nanoseconds / 1e9
    return self._has_joy and age_s <= self.control_timeout_s and
self._joy_control_enabled

# -----
# Motion – stick (analog curve) with D-Pad digital fallback
# -----

def _compute_target_velocity(self, joy: Joy) -> tuple[float, float, bool]:
    """Returns (target_linear, target_angular, stop).

    Car convention: linear.x < 0 → FORWARD, angular.z < 0 → LEFT TURN.
    Joy convention: LY up=-1, RX left=-1 → signs pass through directly.
    """

    linear_speed = LINEAR_GEAR_TABLE[self._linear_gear_index]
    angular_speed = ANGULAR_GEAR_TABLE[self._angular_gear_index]

    # Linear: Left Stick LY, D-Pad V fallback
    ly_raw = self._axis(joy, AXIS_LY)
    if abs(ly_raw) > STICK_DEADZONE:
        ly = self._apply_deadzone(ly_raw, STICK_DEADZONE)
        target_linear = self._curve(ly, LINEAR_CURVE_POWER) * linear_speed
    else:
        dpad_v = self._axis(joy, AXIS_DPAD_V)
        if dpad_v < -0.5:      # D-Pad up → forward (linear.x < 0)
            target_linear = -linear_speed
        elif dpad_v > 0.5:    # D-Pad down → backward
            target_linear = linear_speed
        else:
            target_linear = 0.0

    # Angular: Right Stick RX, D-Pad H fallback
    rx_raw = self._axis(joy, AXIS_RX)
    if abs(rx_raw) > STICK_DEADZONE:
        rx = self._apply_deadzone(rx_raw, STICK_DEADZONE)
        target_angular = self._curve(rx, ANGULAR_CURVE_POWER) *
angular_speed
    else:
        dpad_h = self._axis(joy, AXIS_DPAD_H)
        if dpad_h < -0.5:      # D-Pad left → left turn (angular.z < 0)
            target_angular = -angular_speed
        elif dpad_h > 0.5:    # D-Pad right → right turn
            target_angular = angular_speed
        else:
            target_angular = 0.0

    stop = target_linear == 0.0 and target_angular == 0.0
    return target_linear, target_angular, stop

# -----
# CmdVel timer – target velocity + soft-stop
# -----

def _cmd_timer_callback(self) -> None:
    if not self._joy_control_enabled:

```

```

        return

    dt = self.cmd_publish_period_s
    valid = self._joy_valid()

    if valid:
        target_linear, target_angular, stop =
self._compute_target_velocity(self._latest_joy)
    else:
        target_linear, target_angular, stop = 0.0, 0.0, True

    if valid and not stop:
        # --- Motion active: snap to target (instant response) ---
        self._stop_elapsed = 0.0
        self._current_linear_vel = target_linear
        self._current_angular_vel = target_angular
    else:
        # --- Soft-stop ---
        self._stop_elapsed += dt
        if self._stop_elapsed > self.soft_stop_max_time:
            # Exceeded max stop time → hard stop
            self._current_linear_vel = 0.0
            self._current_angular_vel = 0.0
        else:
            self._current_linear_vel = self._move_toward(
                self._current_linear_vel, 0.0, self.soft_stop_linear * dt
            )
            self._current_angular_vel = self._move_toward(
                self._current_angular_vel, 0.0, self.soft_stop_angular * dt
            )

    twist = Twist()
    twist.linear.x = self._clamp(self._current_linear_vel, -
MAX_LINEAR_SPEED, MAX_LINEAR_SPEED)
    twist.linear.y = 0.0
    twist.angular.z = self._clamp(self._current_angular_vel, -
MAX_ANGULAR_SPEED, MAX_ANGULAR_SPEED)
    self._cmd_vel_pub.publish(twist)

    # -----
    # Servo timer – face buttons + acceleration-limited interpolation
    # -----

    def _compute_servo_direction(self, joy: Joy) -> tuple[int, int]:
        """X → S1 +1, B → S1 -1; Y → S2 -1, A → S2 +1; opposite pair cancels."""
        x_on = self._button(joy, BTN_X) == 1
        b_on = self._button(joy, BTN_B) == 1
        y_on = self._button(joy, BTN_Y) == 1
        a_on = self._button(joy, BTN_A) == 1

        s1 = 1 if (x_on and not b_on) else (-1 if (b_on and not x_on) else 0)
        s2 = -1 if (y_on and not a_on) else (1 if (a_on and not y_on) else 0)
        return s1, s2

    def _update_servo_axis(
        self, axis: ServoAxisState, direction: int,
        direction_sign: float, min_angle: float, max_angle: float, dt: float,
    ) -> None:

```

```

        if direction == 0:
            axis.velocity_dps = 0.0
            return

        target_velocity = float(direction) * direction_sign *
self.servo_speed_dps
        velocity_delta = self.servo_accel_dps2 * dt
        axis.velocity_dps = self._move_toward(axis.velocity_dps,
target_velocity, velocity_delta)
        axis.angle += axis.velocity_dps * dt
        axis.angle = self._clamp(axis.angle, min_angle, max_angle)

        if axis.angle <= min_angle or axis.angle >= max_angle:
            axis.velocity_dps = 0.0

def _servo_timer_callback(self) -> None:
    if not self._joy_control_enabled:
        return

    if self._joy_valid():
        s1_dir, s2_dir = self._compute_servo_direction(self._latest_joy)
    else:
        s1_dir, s2_dir = 0, 0

    dt = self.servo_publish_period_s
    self._update_servo_axis(
        self._servo_s1, s1_dir, SERVO_S1_DIR_SIGN, SERVO_S1_MIN,
SERVO_S1_MAX, dt,
    )
    self._update_servo_axis(
        self._servo_s2, s2_dir, SERVO_S2_DIR_SIGN, SERVO_S2_MIN,
SERVO_S2_MAX, dt,
    )

    servo_msg = Vector3()
    servo_msg.x = float(self._servo_s1.angle)
    servo_msg.y = float(self._servo_s2.angle)
    servo_msg.z = 0.0
    self._servo_angle_pub.publish(servo_msg)

# -----
# Beeper timer
# -----

def _beep_timer_callback(self) -> None:
    beep_msg = UInt16()

    # SELECT toggle beep is independent of joystick enable state,
    # so we process it before the enabled guard.
    if self._joy_toggle_beep_ticks > 0:
        beep_msg.data = 50 # 50 units x 10ms = 500ms pulses
        self._joy_toggle_beep_ticks -= 1
        self._beep_pub.publish(beep_msg)
        if self._joy_toggle_beep_ticks == 0:
            # Send explicit stop after last tick to cut off firmware beep
cleanly
            self._beep_pub.publish(UInt16(data=0))
    return

```

```

if not self._joy_control_enabled:
    return

# Priority: START continuous beep > speed-adjust feedback beep
start_beep = (
    START_BEEP_UNITS
    if self._joy_valid() and self._button(self._latest_joy, BTN_START)
    else 0
)
if start_beep > 0:
    beep_msg.data = start_beep
    self._speed_beep.reset() # suppress pending speed beeps
elif self._speed_beep.active:
    self._speed_beep.tick()
    beep_msg.data = self._speed_beep.value()
else:
    beep_msg.data = 0

self._beep_pub.publish(beep_msg)

```